VERIQTA

Python for DevOps

Build 10 Command-Line Tools

That Solve Real Problems



VERIQTA

Incident Analysis | Deployment Safety | Infrastructure Drift | Reliability (SLO) | Cost Investigation | Backup Recoverability

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

VERIQTA

The Outage Started Before the First Alert

Build a Python CLI that reconstructs the sequence of a production incident from multiple log files.

Project Objectives

- Parse Nginx, systemd, application, and deployment logs.
- Normalize timestamps across time zones and inconsistent formats.
- Correlate request IDs, error messages, restarts, and deployments.
- Distinguish the earliest observed failure from downstream symptoms.
- Output a chronological incident timeline with supporting evidence.

Log Sources

- Nginx Logs
- systemd Logs
- Application Logs
- Deployment Logs

How the Tool Works

Collect Logs → Normalize Timestamps (UTC) → Correlate Events (Request ID, (Rerors, Restarts) → Build Incident Timeline → Evidence Report (JSON/MD)

Expected Output (Example)

```
$ veriqta-incident reconstruct --logs ./incident/ --window 30m
[09:14:02]  Deployment started (v2.8.0)
[09:15:37]  App errors increased (HTTP 500)
[09:16:10]  Nginx upstream timeouts
[09:16:25]  Service restart (systemd)
[09:17:48]  Database connection pool exhausted
            (earliest failure)
```

CLI Usage

```
veriqta-incident reconstruct \
 --logs ./incident/ \
 --window 30m \
 --output report.md
```

Real-World Outcome

- Reconstructs what happened and when.
- Identifies the true starting point of the outage.
- Provides evidence for incident review and postmortem.

When Logs Disagree About What Failed First

Improve the incident reconstruction tool to handle real-world log inconsistencies and validate it with a 502-error scenario.

Key Tasks

- Handle missing timestamps, duplicate events, and clock skew.
- Group related failures without assuming correlation proves causation.
- Rank incident hypotheses using supporting and conflicting evidence.
- Generate JSON and Markdown incident reports.
- Test with a simulated 502-error incident caused by an exhausted database connection pool.

Incident Scenario (Test Case)



Hypothesis Ranking (Example)

Rank	Hypothesis	Evidence	Confidence
1	Database connection pool exhausted	Many "connection timeout" errors, rising DB wait time	0.92
2	Application performance degradation	Increased request latency, thread queue growth	0.74
3	Nginx misconfiguration	No config change, no evidence in Nginx logs	0.18

Output Reports



CLI Command

```
veriqta-incident reconstruct \
 --logs  ./incident/ \
 --window 30m
```

Production Outcome

- Engineers can reconstruct an incident without manually searching thousands of unrelated log entries.

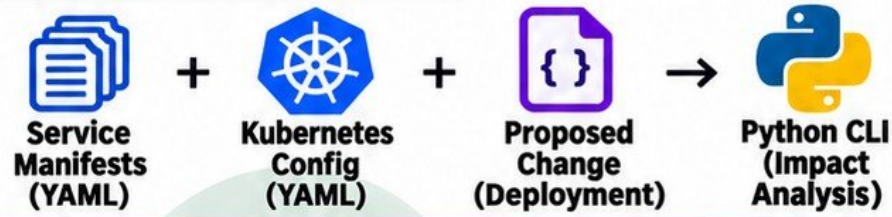
The Deployment Changed One Service but Broke Six Others

Build a CLI that predicts which services could be affected by a proposed deployment.

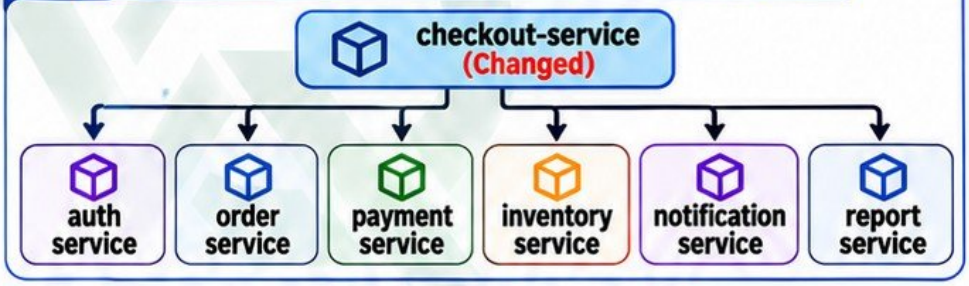
Key Tasks

- Read service dependency manifests and Kubernetes configuration.
- Construct a dependency graph using Python.
- Identify upstream consumers and downstream dependencies.
- Detect changes to shared configuration, Secrets references, and network policies.
- Produce a potential blast-radius report before deployment.

Solution Overview



Service Dependency Graph (Example)



Blast-Radius Report (Output)

Service	Impact	Reason
auth-service	High	Uses shared Secret: app-credentials
order-service	High	Calls checkout API (direct dependency)
payment-service	High	Depends on changed config: payment.yaml
inventory-service	Medium	Network policy affected: allow-checkout
notification-service	Medium	Uses shared ConfigMap: app-config
report-service	Medium	Indirect dependency via order-service

CLI Usage

```
veriqta-blast-radius assess \
 --before old/ \
 --after new/ \
 --env production
```

Expected Outcome

- A pre-deployment blast-radius report that shows which services are likely to be affected and why.

Should This Deployment Be Allowed Into Production?

Evaluate the proposed change, its impact, and operational readiness before it reaches production.

Key Tasks

- Classify changes by potential impact and reversibility.
- Evaluate dependency criticality, redundancy, and rollback readiness.
- Detect missing readiness probes and inadequate PodDisruptionBudgets.
- Generate a machine-readable deployment risk assessment (JSON/YAML).
- Block high-risk deployments using defined policy thresholds.

What the Tool Checks


Dependencies & Criticality


Redundancy & Capacity


Readiness Probes


PDB (Disruption Budget)


Rollback Readiness

Deployment Risk Assessment (Example)

Check	Result	Details
Change Impact	High	Affects 6 dependent services
Rollback Ready	Yes	Previous version available
Readiness Probe	Missing	No *readinessProbe* in new manifest
PodDisruptionBudget	Inadequate	PDB minAvailable: 0
Overall Risk	BLOCK	Above policy threshold (High)

Decision Flow


Proposed Change (YAML)


Risk Checks (Automated)


Assessment (Impact, Rollback, Probes, PDB)



CLI Command

```
veriqta-blast-radius assess \
 --before old/ \
 --after new/
```

Production Outcome

A CI/CD pipeline can detect dangerous configuration changes before they reach production.

The Pod Is Running, but the Application Is Unavailable

Build a Python CLI that collects and correlates Kubernetes troubleshooting evidence.

Key Tasks

- Inspect Deployments, Pods, Services, EndpointSlices, and events.
- Detect selector mismatches and Pods excluded from ready endpoints.
- Investigate CrashLoopBackOff, OOMKilled, and image pull failures.
- Compare requested resources against observed scheduling problems.
- Identify evidence that separates application failure from routing failure.

Kubernetes Evidence Collection

Deployment (Spec & Replicas)	Pods (Status, Events)	Services (Selector, Endpoints)	EndpointSlices (Ready Endpoints)	Events (Timeline & Errors)

Failure Analysis (Examples)

Finding	Evidence	Likely Cause
Pod CrashLoopBackOff	RestartCount: 15, ExitCode: 1	Application crash (e.g., config or code)
Pod OOMKilled	Reason: OOMKilled Memory: 512Mi/512Mi	Insufficient memory (limit too low)
Image Pull Failure	ErrImagePull / ImagePullBackOff	Invalid image or registry access issue
Selector Mismatch	Service has 0 endpoints	Service selector does not match Pod labels

Diagnostic Flow

Collect Cluster Data → Analyze Pods & Events → Check Routing (Services/Endpoints) → Classify Root Cause → Generate Report (App vs Routing)

CLI Command

```
veriqta-k8s diagnose \
--namespace payments \
--deployment api
```

Outcome

A structured, evidence-based report that quickly identifies the root cause and whether the failure is in the application or in routing.

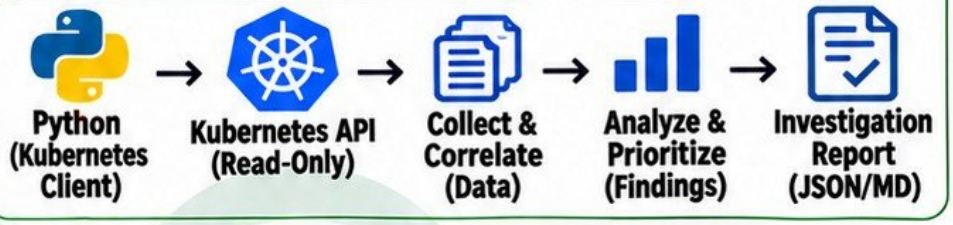
Diagnose the Cluster Without Making the Incident Worse

Safely investigate Kubernetes issues, collect evidence, and produce a prioritized report with confidence levels.

Key Tasks

- Use the Kubernetes Python client with read-only permissions.
- Handle API throttling, RBAC denials, and incomplete cluster responses.
- Correlate events with container status and workload configuration.
- Produce a prioritized investigation report with confidence levels.
- Validate the tool against deliberately broken Kubernetes deployments.

Tool Architecture (Read-Only)



Investigation Coverage



Prioritized Findings (Example)

#	Finding	Severity	Confidence
1	Pod CrashLoopBackOff (api-7d9c)	High	0.95
2	RBAC access denied (list pods)	Medium	0.88
3	API rate limit (throttling by server)	Medium	0.80
4	Incomplete endpoints (selector mismatch)	High	0.92

CLI Command

```
veriqta-k8s diagnose \
  --namespace payments \
  --deployment api
```

Production Outcome

- A reproducible evidence bundle that helps on-call engineers investigate Kubernetes incidents faster and safely.

VERIQTA

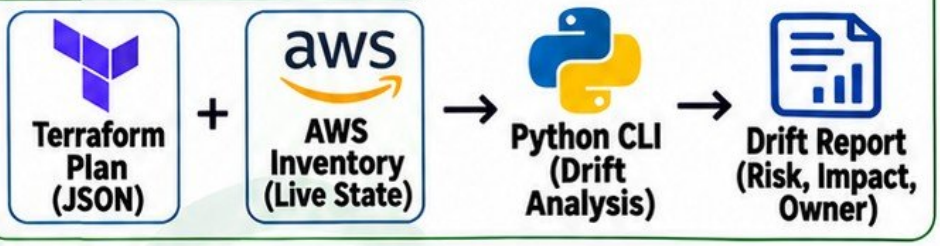
Terraform Says Everything Is Managed. Production Disagrees

Build a Python CLI that detects and explains infrastructure drift between Terraform and live AWS resources.

Key Tasks

- Read Terraform plan JSON and AWS resource inventory.
- Compare security groups, IAM policies, tags, and network settings.
- Detect console-side changes and unmanaged resources.
- Distinguish dangerous drift from expected dynamic state.
- Associate findings with affected applications and resource owners.

Solution Flow



What the Tool Compares



Drift Findings (Example)

Resource	Drift Detected	Risk	Impact
sg-web	Inbound 0.0.0.0/0 added (outside Terraform)	High	Public exposure (API/Portal)
iam-app-role	Policy changed (added admin actions)	High	Excessive permissions
ec2-backup	Not in Terraform (unmanaged resource)	Medium	Cost and compliance risk
vpc-prod	Tag changed: env=dev	Low	Tag drift (non-critical)

CLI Command

```
veriqta-drift inspect \
--environment production \
--plan tfplan.json
```

Production Outcome

A clear, actionable drift report that exposes real risks, explains impact, and links to the affected application and owner.

Not Every Drift Finding Should Trigger Automatic Remediation

Evaluate drift by risk, impact, and reversibility, and propose safe, approved remediation actions.

Key Tasks

- Classify drift by exposure, business impact, and reversibility.
- Detect overly permissive ingress rules and changed IAM trust relationships.
- Separate informational findings from urgent security risks.
- Generate remediation proposals without applying them.
- Build an auditable approval workflow for high-impact corrections.

Drift Risk Classification

Risk Level	Exposure	Business Impact	Reversibility	Action
High	Public / Excessive	Service outage / Data risk	Complex	Require Approval
Medium	Limited / Internal	Performance / Cost / Compliance	Moderate	Review & Plan
Low	Expected / Dynamic	Minimal	Easy	Track (No Action)

Drift Examples

Ingress Rule Changed (0.0.0.0/0)	IAM Trust Relationship Modified	Tag / Capacity Change (Expected)
High Risk (Exposed Service)	High Risk (Privilege Escalation)	Low Risk (Informational)

Approval Workflow

Drift Detected (Scan) → Classify Risk (High/Med/Low) → Propose Remediation (Plan Only) → Approval (Owner/Change Board) → Execute (After Approval)

CLI Command

```
veriqta-drift inspect \
 --environment production \
 --plan  tfplan.json
```

Production Outcome

Teams can investigate infrastructure drift, assess real risk, and propose approved fixes without blindly overwriting legitimate changes.

The Rollback Command Exists, but the Rollback Cannot Work

Build a Python CLI that determines if a production change is safe to release and whether a reliable rollback is possible.

Key Tasks

- Inspect deployment manifests, migration metadata, and release artifacts.
- Verify image immutability and artifact provenance (source and integrity).
- Detect incompatible database migration and rollback combinations.
- Validate health checks, deployment strategy, and recovery prerequisites.
- Identify changes that require a maintenance window or manual approval.

What the Tool Checks

- **Images** (Immutability, Source, Digest)
- **Migrations** (Forward / Rollback Compatibility)
- **Deployment Manifests** (Strategy, Health Checks)
- **Rollback Readiness** (Artifacts, State, Data)
- **Release Policy** (Maintenance or Approval)

Release Assessment (Example)

Check	Result	Details
Image Immutability	Pass	Uses immutable digest (sha256:...)
Migration Compatibility	Fail	DB migration is not safely reversible
Health Checks	Pass	Readiness & liveness probes configured
Deployment Strategy	Warn	Rolling update (no canary)
Rollback Readiness	Warn	Rollback artifact available, but database state not reversible

Decision Flow



CLI Command

```
veriqta-release check \
--change ./release/
--migrations ./migrations/
```

Production Outcome

- A clear release decision (release, approve, or block) with evidence, risks, and rollback readiness.

Build a Release Gate That Understands Operational Risk

Create a release gate that checks operational health, risk, and recoverability before allowing a production deployment.

Key Tasks

- Integrate the release gate with GitHub Actions or a CI/CD pipeline.
- Evaluate error-budget consumption and current incident status.
- Produce PASS, WARN, or BLOCK decisions with clear reasons.
- Require approved exceptions for defined critical findings.
- Simulate a failed deployment where reverting the code cannot reverse the database migration.

System Integration Flow



Release Gate Checks



Decision Matrix

Decision	When It Happens	Example Reason
PASS	Low risk, healthy system, recoverable change	Error budget 85% available, no active incident, migration reversible
WARN	Medium risk or reduced margin	Error budget 60% available, non-critical migration change
BLOCK	High risk, active incident, or non-reversible change	Database migration is irreversible; rollback cannot restore previous state

CLI Command

```
veriqta-release gate \
--release v2.8.0 \
--environment prod
```

Production Outcome

- A release process that checks recoverability and operational risk, not just successful builds and tests.

Every Server Is Healthy, but the Application Cannot Connect

Build a Python CLI to investigate DNS resolution and service connectivity issues, and identify the real root cause.

Key Tasks

- Compare DNS responses from multiple resolvers (e.g., system, public, custom).
- Inspect A, AAAA, CNAME, and SRV records.
- Detect inconsistent responses, stale records, and unexpected TTLs.
- Test TCP and TLS connectivity independently of DNS.
- Distinguish DNS failures from connection refusals, TLS errors, and application timeouts.

Solution Flow

Python (DNS & (Connectivity CLI) → Query DNS (Multiple Resolvers) → Analyze Records (A, AAAA, CNAME, SRV, TTL) → Test Connectivity (TCP & TLS) → Diagnosis Report (Root Cause & Evidence)

DNS & Connectivity Checks

DNS Resolution (Multi-Resolver) | DNS Records (A, AAAA, (CNAME, SRV) | TTL Validation (Stale or (Unexpected) | TCP Check (Port Reachability) | TLS Check (Certificate, Handshake, Expiry) | Result (DNS / TCP / TLS / Application)

Common Findings (Example)

Check	Result	Meaning
DNS (A record)	PASS	Resolves to expected IP (10.0.1.25)
DNS (TTL)	WARN	TTL too high (86,400s) – possible stale entry
TCP (Port 443)	PASS	Port reachable
TLS Handshake	FAIL	Certificate expired – causes connection failure
Overall	FAIL	Application unavailable (TLS error, not DNS)

CLI Command

```
veriqta-dns check \
 --host app.veriqta.com \
 --resolvers 8.8.8.8,1.1.1.1 \
 --port 443 --tls
```

Production Outcome

A clear, evidence-based diagnosis that identifies whether the failure is DNS-related, a network/connection issue, TLS error, or an application timeout.

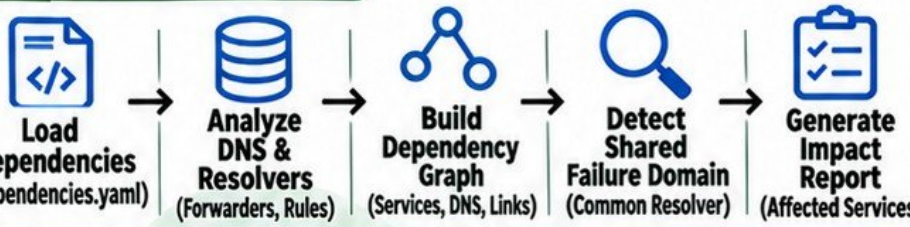
Discover the Shared Dependency That Could Take Down Production

Analyze DNS and service dependencies to uncover shared failure points that can disrupt multiple services.

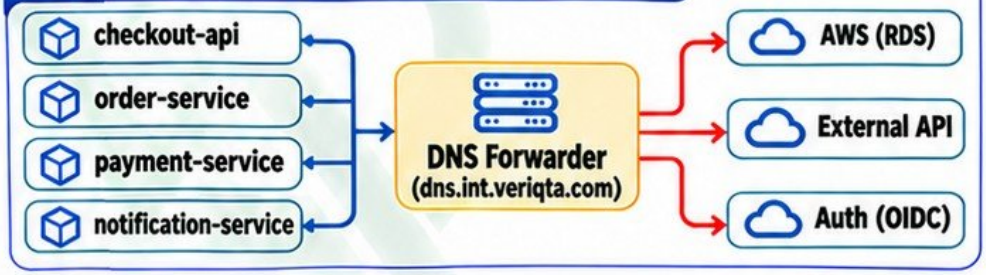
Key Tasks

- Analyze dependency inventories and resolver configurations.
- Detect services relying on the same DNS forwarding endpoints.
- Identify shared failure domains (e.g., DNS, network, provider).
- Produce a dependency graph showing affected workloads.
- Simulate a DNS forwarding failure that disrupts multiple services.

Solution Flow



Dependency Graph (Example)



Failure Simulation (DNS Forwarder Down)

Service	Status (Before)	Status (After)	Reason (Impact)
checkout-api	Healthy	Unavailable	DNS resolution failed
order-service	Healthy	Unavailable	Cannot resolve dependent host
payment-service	Healthy	Unavailable	DNS lookup timeout
notification-service	Healthy	Degraded	Partial failures (external API)

CLI Command

```
veriqta-dependency diagnose \
--service checkout-api \
--config dependencies.yaml
```

Production Outcome

Engineers can investigate hidden DNS and connectivity dependencies before restarting healthy applications unnecessarily.

The Monitoring Dashboard Is Green While Customers Are Failing

Build a Python CLI that measures real service reliability using customer-facing metrics, SLOs, and error budgets.

Key Tasks

- Retrieve Prometheus metrics through its HTTP API.
- Calculate availability and latency service-level indicators (SLIs).
- Define SLO targets (e.g., 99.9% uptime, latency < 300 ms) and error budgets.
- Detect rapid error-budget consumption using multiple time windows (5m, 1h, 24h).
- Distinguish infrastructure health from customer-visible service health.

Solution Flow

Prometheus (HTTP API) → Collect Metrics (Requests, (Errors, Latency) → Compute SLIs (Availability, Latency) → Evaluate SLOs (Targets & Error Budget) → Detect & Report (Health, Burn Rate, Customer Impact)

SLO & Error Budget Example

Indicator	SLO Target	Current (24h)	Status
Availability (Uptime)	≥ 99.9%	99.72%	BREACH
Latency (p95)	≤ 300 ms	420 ms	BREACH
Error Rate	≤ 0.1%	0.28%	BREACH
Error Budget (30d)	0.1% (Budget)	72% Consumed	AT RISK

Infrastructure vs Customer Health

Infrastructure Health (Internal)

- CPU: Normal (28%)
- Memory: Normal (54%)
- Pods: Healthy (3/3)
- Nodes: Healthy (3/3)

✓ GREEN

≠

Customer-Visible Health (External / Real Experience)

- Uptime: 99.72% (below SLO)
- p95 Latency: 420 ms (above target)
- Error Rate: 0.28% (above SLO)
- Error Budget: 72% consumed

✕ FAILING (Customer Impact)

CLI Command

```
veriqta-slo evaluate \
 --service checkout-api \
 --window 5m,1h,24h
 --slo 99.9,300ms
```

Production Outcome

A clear, SLO-based view of service reliability that detects customer impact early, even when the infrastructure dashboard looks healthy.

Stop Paging Engineers for Problems That Do Not Require Action

Build a Python CLI that analyzes alerts, SLOs, and historical data to reduce noise and focus on real customer impact.

Key Tasks

- Analyze historical alert frequency and duration (e.g., last 7–30 days).
- Identify noisy alerts, duplicate notifications, and flapping conditions (rapid on/off).
- Compare alert events with actual SLO violations (availability, latency, error rate).
- Evaluate proposed alert thresholds against historical data (false-positive reduction).
- Generate evidence-based recommendations (e.g., adjust threshold, add delay, or group alerts) without automatically changing rules.

Solution Flow

Load Alert History & SLO Data → Analyze Patterns (Frequency, Duration, Flapping) → Compare with SLOs (Real Customer Impact) → Evaluate Thresholds (Using Historical Data) → Generate Recommendations (With Evidence)

Alert Analysis Summary (Example)

Alert Rule	Count (24h)	SLO Impact	Type	Recommendation
High CPU > 80%	120	No	Noisy	Raise to 90% or add 5m delay
5xx Error Rate	18	Yes	Real	Keep (customer impacting)
Memory > 75%	64	No	Flapping	Add hysteresis (e.g., 5m)
Pod Restart	36	No	Duplicate	Group with existing alert

Key Metrics (24h Window)

Total Alerts	Noisy / Duplicate	SLO Violations	Actionable Alerts
238	156 (65.5%)	2	82 (34.5%)

CLI Command

```
veriqta-slo analyze \
--service checkout \
--window 24h
```

Production Outcome

A cleaner, evidence-backed alerting setup that reduces unnecessary pages and focuses engineers on real customer-impacting incidents.

The AWS Bill Doubled, but Nobody Deployed Anything

Build a Python CLI that analyzes AWS spending, finds the cause of unexpected cost increases, and links it to resource and infrastructure changes.

Key Tasks

- Retrieve AWS Cost Explorer data using Boto3.
- Compare current spending against historical baselines (e.g., last 7, 30, 90 days).
- Break down costs by account, service, region, and tags (e.g., environment, team, project).
- Identify potential cost drivers (NAT Gateway, data-transfer, logging, and compute).
- Correlate billing anomalies with resource inventory and infrastructure changes (e.g., new resources, config updates).

Solution Flow

AWS Cost Explorer (Boto3) → Collect & Normalize Cost Data → Compare with Baseline (Trend & Variance) → Identify Cost Drivers (NAT, Data, Logs, Compute) → Generate Investigation Report (Root Cause & Recommendations)

Cost Increase Analysis (Example)

Service	Last 30 Days	This 30 Days	Change	Potential Cause
NAT Gateway	$120.00	$358.00	+198%	Higher data egress or new traffic
Data Transfer	$60.00	$145.00	+142%	Increased outbound traffic (region/Internet)
CloudWatch Logs	$40.00	$110.00	+175%	Higher log ingestion/ retention
EC2 (Compute)	$200.00	$340.00	+70%	Larger instance count or runtime
Total	**$420.00**	**$953.00**	**+127%**	**Overall spend doubled**

Spending Insights & Root Cause

Likely Cost Drivers

- NAT Gateway usage ↑
- Data transfer (egress) ↑
- CloudWatch log volume ↑
- Increased compute runtime

Investigation Output

- Cost change: +127% ($420 → $953)
- Primary driver: NAT Gateway (+198%)
- Related change: new private subnets and higher outbound traffic (no new deploy)
- Recommendation: review egress, NAT usage, log retention, and idle resources

CLI Command

```
veriqta-cost investigate \
  --period 30d \
  --account prod \
  --output report
```

Production Outcome

A clear, evidence-based cost investigation that explains why spending increased and what to review, without guessing.

Find the Cost Reduction That Will Not Break Production

Build a Python CLI to identify safe cloud cost optimizations, estimate savings, and assess reliability impact before any changes are made.

Key Tasks

- Distinguish idle resources from intentionally reserved capacity (e.g., HA, failover, scaling buffer).
- Identify potentially oversized instances using utilization evidence (CPU, memory, network).
- Evaluate the reliability impact of proposed savings (SLOs, redundancy, risk).
- Estimate potential cost savings with clear assumptions (e.g., instance size, hours, region).
- Generate a reviewable cost-reduction report without deleting resources.

Solution Flow



Resource Analysis (Example)

Resource	Type	Utilization (Avg)	Finding	Action (Proposed)	Est. Savings (14d)
web-ec2-01	EC2	CPU 8% | Mem 18%	Oversized	Downsize (t3.large → t3.medium)	$42.00
batch-ec2-02	EC2	CPU < 2% | Mem 5%	Idle	Stop or schedule (non-prod)	$38.50
nat-gw-1	NAT GW	Low egress (<1 GB/day)	Underused	Review / consolidate (NAT or VPC endpoints)	$65.00
rds-db-01	RDS	CPU 12% | Mem 28%	Right-sized	Keep (production, within SLO)	$0.00
Total Potential Savings (14 days)					**$145.50**

Reliability & Savings Assessment

Proposal	Reliability Impact	Risk Level	Recommendation
Downsize EC2 (web-ec2-01)	No impact (utilization < 20%, SLO met)	Low	Proceed (after review)
Stop idle instance (batch-ec2-02)	No production impact (non-prod workload)	Low	Proceed (schedule)
Review NAT Gateway (nat-gw-1)	Possible impact if shared (verify dependencies)	Medium	Review & test first
Keep RDS (rds-db-01)	Production database (critical, within SLO)	None	No change

CLI Command

```
veriqta-cost investigate \
 --account production \
 --period 14d
```

Production Outcome

Engineers can explain unexpected cloud costs and propose safe reductions without sacrificing reliability.

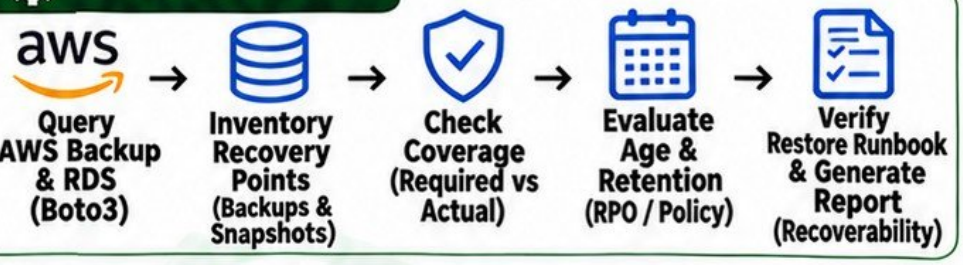
The Backup Job Succeeded. Can the Business Actually Recover?

Build a Python CLI that audits backup recoverability, coverage, and restoration readiness for business continuity.

Key Tasks

- Inventory AWS Backup recovery points and RDS snapshots.
- Compare backup coverage against protected-resource requirements (e.g., RDS, EC2, EFS).
- Calculate backup age and potential recovery-point exposure (RPO risk).
- Identify missing coverage, retention gaps, and inaccessible recovery artifacts.
- Verify the existence of documented restoration procedures (runbooks).

Solution Flow



Backup Assessment (Example)

Resource	Coverage	Latest Backup	Backup Age	Retention	Status
prod-db (RDS)	Yes	Nov 4, 2026 02:00	14h	30 days	Healthy
app-ec2	Yes	Nov 3, 2026 23:30	18h	7 days	At Risk
app-data (EFS)	Yes	Nov 4, 2026 01:45	15h	30 days	Healthy
analytics-db	No	–	–	–	Missing

Findings & Risks

- 1 critical resource without backup (analytics-db).
- 1 resource near RPO limit (app-ec2: 18h > 12h target).
- Retention gap (app-ec2: 7 days < 30 days policy).
- No verified restore runbook for analytics-db.

Recommendations

- Enable backup for analytics-db (align with policy).
- Increase app-ec2 retention to 30 days.
- Validate restore procedure for all critical resources.
- Run a periodic recovery test (restore to a test environment).

CLI Command

```
veriqta-backup audit \
  --account production \
  --resource all
  --output report
```

Production Outcome

A verifiable backup and recovery posture that confirms the business can actually restore when needed, not just that backups completed.

Turn Disaster Recovery Claims Into Testable Evidence

Build a Python CLI that validates disaster recovery through restoration drills, RTO/RPO checks, dependency verification, and an evidence-based report.

Key Tasks

- Analyze restoration drill results from isolated test environments.
- Measure recovery time against defined RTO requirements.
- Compare restored data currency against RPO requirements.
- Detect missing recovery dependencies (e.g., IAM access, DNS, encryption keys).
- Generate an evidence-based recovery readiness report.

Solution Flow



Recovery Drill Results (Example)

Component	RTO Target	Actual RTO	RPO Target	Actual Data Gap	Status
payments-api	≤ 30 min	18 min	≤ 5 min	3 min	PASS
payments-db	≤ 60 min	52 min	≤ 5 min	4 min	PASS
cache (Redis)	≤ 15 min	27 min	≤ 5 min	8 min	FAIL
dependencies	—	—	—	IAM, DNS OK Key: MISSING	FAIL

Findings

- RTO breach: cache (27 min > 15 min).
- RPO breach: cache data gap (8 min > 5 min).
- Missing recovery dependency: encryption key not available in test environment.

Recommendations

- Restore and validate encryption key access for DR.
- Optimize cache recovery to meet 15 min RTO.
- Re-run the DR drill after fixes and update the evidence report.
- Keep DR results, logs, and timestamps for audit and compliance.

CLI Command

```
veriqta-recovery audit \
 --workload payments \
 --policy recovery.yaml
```

Production Outcome

Teams can distinguish a successful backup from demonstrated recoverability. A backup inventory alone must never be treated as proof that restoration will succeed.

Automation Can Stop an Incident or Turn It Into a Regional Outage

Build a Python CLI that safely proposes and executes approved incident-response actions, models impact, and enforces guardrails to prevent wider outages.

Key Tasks

- Ingest incident signals and workload dependency data (e.g., alerts, metrics, service map).
- Evaluate containment options (pause deployments, reduce traffic, isolate a worker pool).
- Model the impact of each action on dependent services (e.g., affected SLOs, traffic, capacity).
- Enforce approval, scope, and environment restrictions (e.g., prod-only, service allowlist).
- Require dry-run previews, timeouts, and idempotency safeguards (to prevent duplicate or destructive actions).

Solution Flow

Ingest Incident Signals (Alerts, Metrics) → Load Dependencies (Service Map) → Evaluate Actions (Containment Options) → Model Impact (SLO, Traffic, Dependencies) → Enforce Guardrails (Approval, Scope, Env) → Execute Safely (Dry-Run, Timeout, Idempotent)

Containment Options & Impact (Example)

Action (Command)	Purpose	Estimated Impact	Risk	Approval
Pause Deployment	Stop new changes	No customer impact	Low	Required
Reduce Traffic (50%)	Lower load quickly	Slight latency ↑	Medium	Required
Isolate Worker Pool	Remove failing nodes	Partial feature impact	Medium	Required
Scale Up (+2 pods)	Add capacity	Restore performance	Low	Allowed

Safety Controls

- Dry-run first (show planned changes and estimated impact).
- Approval required for production changes.
- Environment and service scope restrictions.
- Timeouts for all actions (e.g., 5–10 minutes).
- Idempotency (safe to re-run without side effects).

Example Output (Dry-Run)

```
$ veriqta-incident respond \
 --incident INC-5821 --action reduce-traffic\
 --service checkout --percent 50 --dry-run
```

```
DRY-RUN RESULT
Action: Reduce traffic to checkout (50%)
Estimated impact:
  - Requests: ~2,400 → ~1,200 rps
  - Latency: +80 ms (estimated)
  - Affected services: payment-api, orders
Approval: Required  |  Timeout: 10m
Idempotent: Yes
```

CLI Command

```
veriqta-incident respond \
  --incident INC-5821
  --action isolate-pool
  --service workers --dry-run
```

Production Outcome

Controlled automation that contains incidents quickly, with safety, approval, and impact awareness, reducing the risk of a local issue becoming a regional outage.

Final Production Challenge: Defend Your Automation During a Live Incident

Scenario: A payment platform experiences rising latency, database connection exhaustion, and repeated application retries immediately after a deployment.

Your Tasks

- Correlate the incident evidence (metrics, logs, traces, and events).
- Determine whether rollback is safe (risk, data integrity, impact).
- Identify services at risk from cascading failures (e.g., payments, orders, notifications).
- Select a containment strategy that preserves payment correctness (e.g., reduce traffic, scale DB, limit retries).
- Execute approved actions in a controlled simulation (with dry-run and approval).
- Verify recovery using application health and customer-facing SLOs (latency, error rate, availability).
- Generate a complete audit trail and incident report (timeline, actions, impact, outcome).

Incident Context (Evidence)



Rising Latency
(p95: 1.2s → 4.8s)
+300%



DB Connection Exhaustion
(1,000 / 1,000)
100%



Repeated Application Retries
(↑ retry rate: 8% → 42%)
+425%

Investigation & Decision Flow



Analyze Evidence (Metrics, Logs, Traces) →



Assess Impact (Affected Services) →



Evaluate Rollback (Safe or Not) →



Choose Containment (Reduce, Scale, Isolate) →



Execute Approved Action (Dry-Run → Run) →



Verify Recovery (SLOs & Health)

Containment Options

Option	Purpose	Risk
Rollback to previous version	Remove problematic changes	Low
Reduce traffic (50%)	Stabilize DB connections	Low
Scale database (+ capacity)	Clear connection pressure	Medium
Limit retries (1–2)	Prevent retry amplification	Low

Expected Outcome

- Latency returns to ≤ 800 ms (p95).
- DB connection utilization drops to < 70%.
- Error rate returns to ≤ 0.1%.
- Payment correctness preserved (no duplicate or lost transactions).
- SLOs back within target (≥ 99.9% availability).

CLI Command

```
veriqta-contain plan \
--incident INC-2048 \
--dry-run
```

Final Production Outcome

A controlled incident-response CLI that demonstrates how to combine investigation, risk assessment, authorization, execution, and recovery verification.